باسمه تعالی

آزمایشگاه طراحی سیستم‌های دیجیتال

دانشکده مهندسی کامپیوتر دانشگاه صنعتی شریف

تابستان ۱۴۰۳

استاد:

دکتر محسن انصاری

مهندس محمدرضا حسینی

اعضای گروه:

سعید فراتی کاشانی – ۴۰۱۱۰۶۲۹۹

امیرحسین صوری – ۴۰۱۱۰۶۱۸۲

محمدامین عباس‌فر – ۴۰۱۱۰۶۲۱۱

فهرست عناوین

موضوع آزمایش ۳
شرح کلی آزمایش ۳
پیاده‌سازی Transmitter ۳
ورودی‌ها ۳
خروجی‌ها ٤
طراحی Transmitter ٤
پیاده‌سازی Receiver ٦
ورودی‌ها ٦
خروجی‌ها ٦
طراحی Receiver ٦
پیاده‌سازی UART ۸
ورودی‌ها ۸
خروجی‌ها ۸
طراحی UART ۹
برنامه‌ی محک (TestBench) ۱۰

موضوع آزمایش

UART (Universal Asynchronous Receiver Transmitter)

شرح کلی آزمایش

۸-۲ آزمایش هفتم: UART

هدف از انجام این آزمایش طراحی یک Universal Asynchronous Receiver Transmitter (UART) می باشد. در قسمت ارسال کننده این دستگاه هربار یک کد ۷بیتی ASCII بصورت سریال ارسال می گردد. در ابتدا یک بیت شروع (Start)، سپس یک بیت توازن (Parity) و بعد ۷ بیت داده ارسال می شوند. در انتها نیز حداقل یک بیت خاتمه (Stop) ارسال می شود (در مجموع ۱۰ بیت).

در قسمت گیرنده نیز پس از دریافت بیت شروع (Start)، ۸ بیت مربوط به داده و توازن (Parity) بصورت سریال دریافت شده و در یک ثبات (Register) ۸ بیتی ذخیره می شود.

با اتصال دو واحد از این واحدهای UART به یکدیگر نشان دهید که طراحی شما مبادله داده را به درستی انجام می‌دهد.

برای پیاده‌سازی UART، ابتدا Transmitter و Receiver باید تکمیل شوند. سپس در یک ماژول کلی آن‌ها را در کنار یکدیگر چیده‌ایم.

پیاده‌سازی Transmitter

برای پیاده‌سازی فرستنده باید وقتی سیگنال شروع را دریافت کردیم، یک صفر ارسال کنیم تا دستگاه دیگر شروع را تشخیص دهد (مقدار دیفالت ارسالی خارج از بازه‌ی مورد نظر یک است). سپس ۷ بیت داده و یک بیت parity و در نهایت یک بیت ۱ برای اعلام اتمام موفقیت‌آمیز باید ارسال شوند.

ورودی‌ها

clk: سیگنال کلاک

reset: سیگنال ریست که به صورت پایین‌رونده اعمال می‌شود.

start: سیگنال شروع ارسال دیتا

data_in: دیتای ۷ بیتی که بیت به بیت به سوی دستگاه دیگر ارسال می‌شود.

خروجی‌ها

tx: خروجی سریال transmitter که توسط receiver دستگاه دیگر دریافت می‌شود.

busy: سیگنالی که هنگام انتقال یک داده فعال است و قبل و بعد از آن غیرفعال می‌شود.

طراحی Transmitter

کد وریلاگ مربوط به Transmitter به پیوست ارسال می‌شود؛ آن را می‌توانید در ادامه ببینید:

```
module transmitter (
    input clk, reset, start,
    input [6:0] data_in,
    output reg tx, busy
);

    reg [3:0] bit_counter;
    reg [7:0] tx_shift_reg;
    reg parity_bit, tx_busy;

    always @(posedge clk or negedge reset) begin
        if (!reset) begin
            bit_counter <= 0;
            tx_shift_reg <= 0;
            parity_bit <= 0;
            tx <= 1;
            busy <= 0;
            tx_busy <= 0;
        end else if (start && !tx_busy) begin
            tx_shift_reg <= {1'b0, data_in};
            parity_bit <= ^data_in;
            tx_busy <= 1;
            busy <= 1;
            bit_counter <= 0;
        end else if (tx_busy) begin
            case (bit_counter)
                4'd0: tx <= 0;
                4'd1: tx <= tx_shift_reg[0];
                4'd2: tx <= tx_shift_reg[1];
                4'd3: tx <= tx_shift_reg[2];
                4'd4: tx <= tx_shift_reg[3];
```

```
                4'd5: tx <= tx_shift_reg[4];
                4'd6: tx <= tx_shift_reg[5];
                4'd7: tx <= tx_shift_reg[6];
                4'd8: tx <= parity_bit;
                4'd9: tx <= 1;
                default: begin
                    tx <= 1;
                    tx_busy <= 0;
                    busy <= 0;
                end
            endcase
            bit_counter <= bit_counter + 1;
        end
    end

endmodule
```

کد ماژول Transmitter را بخش به بخش بررسی می‌کنیم:

در ابتدا ورودی‌های تک‌بیتی clk، reset، start، ورودی ۷ بیتی data_in و خروجی‌های تک‌بیتی tx و busy تعریف شده‌اند. سپس رجیسترهای ۴ بیتی bit_counter، ۸ بیتی tx_shift_reg و تک‌بیتی‌های parity_bit و tx_busy را تعریف کرده‌ایم که به ترتیب برای تشخیص استیت فعلی، ذخیره‌سازی دیتای ارسالی و parity، ذخیره‌سازی parity محاسبه‌شده و اعلام وضعیت این‌که transmitter در حال ارسال داده است یا خیر در نظر گرفته شده‌اند.

سپس در بلاک always، در صورت فعال شدن reset، همه‌ی متغیرها ریست شده و مقدار دیفالت را به خود می‌گیرند. در غیر این صورت اگر سیگنال شروع فعال شده‌باشد و transmitter در حین ارسال داده نباشد، سیگنال‌های مربوطه را مقداردهی می‌کند، دیتای مورد نیاز را در شیفت‌رجیستر می‌ریزد و بیت parity را برای داده‌ی ورودی محاسبه می‌کند.

در کلاک‌هایی که سیستم در حال ارسال داده است نیز با توجه به این‌که در کدام استیت هستیم، بیت مناسب را در tx خروجی می‌دهیم و پس از هر کلاک مقدار bit_counter را یکی زیاد می‌کنیم.

پیاده‌سازی Receiver

برای پیاده‌سازی گیرنده باید وقتی بیت صفر را در ورودی تشخیص دادیم، فرایند دریافت داده را شروع کنیم و مطابق با سیاست توافق‌شده عمل کنیم؛ در واقع باید ۷ بیت داده و یک بیت parity را دریافت کنیم.

ورودی‌ها

clk: سیگنال کلاک

reset: سیگنال ریست که به صورت پایین‌رونده اعمال می‌شود.

rx: ورودی سریال receiver که توسط transmitter دستگاه دیگر ارسال می‌شود.

خروجی‌ها

data_out: دیتای ۷ بیتی که بیت به بیت از طرف دستگاه دیگر ارسال می‌شود.

parity_error: در صورت عدم مطابقت بیت parity و ایجاد اخلال در ارسال داده، فعال می‌شود.

طراحی Receiver

کد وریلاگ مربوط به Receiver به پیوست ارسال می‌شود؛ آن را می‌توانید در ادامه ببینید:

```
module receiver (
    input clk, reset, rx,
    output reg [6:0] data_out,
    output reg parity_error);

    reg [3:0] bit_counter;
    reg [7:0] rx_shift_reg;
    reg rx_busy, parity_bit;

    always @(posedge clk or negedge reset) begin
        if (!reset) begin
            bit_counter <= 0;
            rx_shift_reg <= 0;
            data_out <= 0;
            rx_busy <= 0;
            parity_error <= 0;
```

```
        end else if (!rx_busy && rx == 0) begin
            rx_busy <= 1;
            bit_counter <= 1;
        end else if (rx_busy) begin
            case (bit_counter)
                4'd1: rx_shift_reg[0] <= rx;
                4'd2: rx_shift_reg[1] <= rx;
                4'd3: rx_shift_reg[2] <= rx;
                4'd4: rx_shift_reg[3] <= rx;
                4'd5: rx_shift_reg[4] <= rx;
                4'd6: rx_shift_reg[5] <= rx;
                4'd7: rx_shift_reg[6] <= rx;
                4'd8: parity_bit <= rx;
                4'd9: begin
                    rx_busy <= 0;
                    data_out <= rx_shift_reg[6:0];
                    parity_error <= (parity_bit != ^rx_shift_reg[6:0]);
                end
            endcase
            bit_counter <= bit_counter + 1;
        end
    end

endmodule
```

کد ماژول Receiver را بخش به بخش بررسی می‌کنیم:

در ابتدا ورودی‌های تک‌بیتی clk، reset و rx، خروجی ۷ بیتی data_out و خروجی تک‌بیتی parity_error تعریف شده‌اند. همچنین رجیسترهای ۴ بیتی bit_counter، ۸ بیتی rx_shift_register و تک‌بیتی rx_busy و parity_bit را به ترتیب برای تشخیص استیت فعلی، ذخیره‌ی داده و بیت parity، وضعیت این که گیرنده در حال دریافت داده است یا خیر و همچنین ذخیره‌ی بیت parity محاسبه‌شده در فرستنده در نظر گرفته‌ایم.

در بلاک always اگر reset فعال شده‌باشد، همه‌ی سیگنال‌ها به مقدار دیفالت خود برمی‌گردند. در غیر این‌صورت اگر گیرنده در حین دریافت داده نباشد و در ورودی سریال خود بیت صفر را تشخیص دهد، وارد حالت دریافت داده می‌شود و bit_counter را روی ۱ تنظیم می‌کند تا از کلاک بعدی شروع به دریافت data کند.

در حالت دریافت داده نیز با توجه به استیت فعلی، ورودی سریالی در جایگاه مناسب از شیفت‌رجیستر ذخیره می‌شود. پس از اتمام فرایند دریافت بیت نیز بیت parity با توجه به داده‌ی دریافت‌شده محاسبه می‌شود و در صورت مغایرت آن با بیت parity محاسبه‌شده در transmitter، سیگنال parity_error فعال می‌شود.

در انتهای هر کلاک نیز مقدار bit_counter یکی زیاد می‌شود.

پیاده‌سازی UART

حالا نوبت آن رسیده‌است تا با کنار هم قرار دادن یک transmitter و یک receiver ماژول UART مورد نظر را تشکیل دهیم.

ورودی‌ها

clk: سیگنال کلاک UART

reset: سیگنال ریست که به صورت پایین‌رونده اعمال می‌شود.

tx_start: سیگنال شروع ارسال دیتا توسط فرستنده

rx: ورودی سریال receiver که توسط transmitter دستگاه دیگر ارسال می‌شود.

tx_data: دیتای ۷ بیتی که بیت به بیت توسط فرستنده به سوی دستگاه دیگر ارسال می‌شود.

خروجی‌ها

tx: خروجی سریال transmitter که توسط receiver دستگاه دیگر دریافت می‌شود.

tx_busy: سیگنالی که هنگام انتقال یک داده از فرستنده فعال است و قبل و بعد از آن غیرفعال می‌شود.

rx_parity_error: در صورت عدم مطابقت بیت parity هنگام دریافت داده توسط گیرنده و ایجاد اخلال در ارسال داده، فعال می‌شود.

rx_data: دیتای ۷ بیتی که بیت به بیت از طرف دستگاه دیگر ارسال می‌شود و توسط گیرنده دریافت می‌شود.

طراحی UART

کد وریلاگ مربوط به UART به پیوست ارسال می‌شود؛ آن را می‌توانید در ادامه ببینید:

```
module UART (
    input clk, reset, tx_start, rx,
    input [6:0] tx_data,
    output tx, tx_busy, rx_parity_error,
    output wire [6:0] rx_data);

    transmitter transmitter (
        .clk(clk),
        .reset(reset),
        .data_in(tx_data),
        .start(tx_start),
        .tx(tx),
        .busy(tx_busy)
    );

    receiver receiver (
        .clk(clk),
        .reset(reset),
        .rx(rx),
        .data_out(rx_data),
        .parity_error(rx_parity_error)
    );

endmodule
```

پس از تعریف ورودی‌ها و خروجی‌های مذکور یک نمونه از ماژول‌های Transmitter و Receiver ساخته‌ایم و سیگنال‌های مربوطه را به آن‌ها متصل کرده‌ایم.

برنامه‌ی محک (TestBench)

```
module TB;

    reg clk, reset, tx_start;
    reg [6:0] tx_data;
    wire tx, tx_busy, rx_parity_error;
    wire [6:0] rx_data;

    UART uart (
        .clk(clk),
        .reset(reset),
        .tx_data(tx_data),
        .tx_start(tx_start),
        .rx(tx),
        .tx(tx),
        .tx_busy(tx_busy),
        .rx_data(rx_data),
        .rx_parity_error(rx_parity_error)
    );

    always #5
        clk = ~clk;

    initial begin
        clk = 1'b0;
        reset = 1'b0;
        tx_data = 7'b0;
        tx_start = 1'b0;

        #20;
        reset = 1'b1;

        #20;
        tx_data = 7'b000111;
        tx_start = 1'b1;
        #20;
        tx_start = 1'b0;

        wait(~tx_busy);

        #100;

        if ((rx_data == tx_data) && !rx_parity_error) begin
            $display("Test passed: Data received correctly");
        end else begin
            $display("Test failed: Data mismatch or parity error");
        end

        #50;
        $finish;
    end

    initial begin
        $monitor("%3.t: reset=%b, tx_start=%b, tx_data=%b,
        tx=%b, rx_data=%b, rx_parity_error=%b",
        $time, reset, tx_start, tx_data, tx, rx_data, rx_parity_error);
    end

endmodule
```

در تست‌بنچ پس از تعریف رجیسترها و سیم‌های مربوطه نمونه‌ای از ماژول UART می‌سازیم و ورودی‌ها و خروجی‌ها را تنظیم می‌کنیم. سیگنال Clk هر ۵ واحد زمانی عوض می‌شود.

تست‌بنچ به این صورت است که واحدهای فرستنده و گیرنده‌ی یک UART را به یکدیگر متصل می‌کند و داده‌ای را از فرستنده به گیرنده ارسال می‌کند. علاوه‌بر مشخص‌بودن سیگنال‌های مربوطه در انتهای فعالیت گیرنده، با توجه به سیگنال parity_error پیغام مربوطه نیز نمایش داده می‌شود.

نتیجه‌ی مانیتور تست‌بنچ را در زیر می‌بینید:

```
  0: reset=0, tx_start=0, tx_data=0000000, tx=1, rx_data=0000000, rx_parity_error=0
 20: reset=1, tx_start=0, tx_data=0000000, tx=1, rx_data=0000000, rx_parity_error=0
 40: reset=1, tx_start=1, tx_data=0000111, tx=1, rx_data=0000000, rx_parity_error=0
 55: reset=1, tx_start=1, tx_data=0000111, tx=0, rx_data=0000000, rx_parity_error=0
 60: reset=1, tx_start=0, tx_data=0000111, tx=0, rx_data=0000000, rx_parity_error=0
 65: reset=1, tx_start=0, tx_data=0000111, tx=1, rx_data=0000000, rx_parity_error=0
 95: reset=1, tx_start=0, tx_data=0000111, tx=0, rx_data=0000000, rx_parity_error=0
135: reset=1, tx_start=0, tx_data=0000111, tx=1, rx_data=0000000, rx_parity_error=0
155: reset=1, tx_start=0, tx_data=0000111, tx=1, rx_data=0000111, rx_parity_error=0
Test passed: Data received correctly
TB.v:49: $finish called at 305 (1s)
```

همان‌طور که می‌دانید هر کلاک ۱۰ واحد زمانی است و در نتایج تست‌بنچ نیز مشخص است که در زمان ۶۰، سیگنال شروع فعال شده‌است، سپس از زمان ۶۵ تا ۹۵ (یعنی به مدت ۳ کلاک) بیت ۱ و از زمان ۹۵ تا ۱۳۵ (یعنی به مدت ۴ کلاک) بیت ۰ ارسال شده‌است. در نتیجه به هدف خود که ارسال داده‌ی 0000111 بود، رسیدیم.